

Project 2: Robust Object Matching Under Motion and Noise

Group Member: Sahej, Harkirat, Trithep

1. Introduction

Tracking the same object across video frames is a challenging computer vision problem due to factors such as motion blur, noise, illumination changes, and object movement. Feature-based tracking methods are widely used because they are robust to scale and rotation changes.

In this project, an **advanced video object tracking system** is implemented using Python and OpenCV. The system combines **image filtering, SIFT feature matching, Harris corner detection, and optical flow validation** to reliably match and track objects across consecutive video frames.

2. Project Objective

The objective of this project is:

To match and track the same object across video frames under blur, noise, and motion conditions.

This is achieved by detecting reliable feature points, matching them between frames, and validating their motion consistency.

3. Tools and Technologies Used

- **Programming Language:** Python
- **Libraries:**
 - OpenCV (computer vision operations)
 - NumPy (numerical computation)
 - Dataclasses and typing modules (code structure and clarity)
- **Input:** Video file ([Gorilla Walks.mp4](#))
- **Output:** Real-time visualization of detected features, matches, and optical flow vectors

4. System Design Overview

The system processes video frames sequentially and applies a multi-stage pipeline:

1. Frame acquisition from video
2. Grayscale conversion
3. Gaussian filtering
4. Harris corner detection
5. SIFT feature detection
6. Feature filtering using Harris corners
7. Feature matching with previous frame
8. Optical flow consistency validation
9. Visualization and statistics update

5. Implementation Details

5.1 Configuration Setup

All system parameters are stored in a configuration class (`TrackerConfig`).

This includes:

- Feature detection parameters
- SIFT matching thresholds
- Harris corner detection settings
- Optical flow parameters

Using a configuration class makes the system flexible and easy to tune.

5.2 Image Preprocessing

Each frame is converted to grayscale and smoothed using a **Gaussian blur**. This step reduces noise and improves the stability of feature detection.

5.3 Harris Corner Detection

Harris corner detection is applied to identify strong corner points in the frame.

- The Harris response is computed and thresholded
- A binary mask is generated to represent corner locations
- These corners are used later to filter weak SIFT keypoints

This ensures that only geometrically meaningful features are retained.

5.4 SIFT Feature Detection

SIFT is used to detect keypoints and compute descriptors for each frame.

- Keypoints represent distinctive image locations
- Descriptors encode local appearance around each keypoint

SIFT is chosen because it is robust to scale, rotation, and illumination changes.

5.5 Harris-Based Feature Filtering

Detected SIFT keypoints are filtered using the Harris corner mask.

- Only SIFT keypoints near Harris corners are retained

- Weak and unstable features are removed

This significantly improves matching accuracy and reduces false positives.

5.6 Feature Matching

Feature matching is performed between the current and previous frames using:

- Brute-Force matcher with L2 distance
- K-Nearest Neighbors ($k = 2$)
- Lowe's ratio test

Only high-quality matches that pass the ratio test are accepted.

5.7 Optical Flow Validation

To ensure temporal consistency, matched features are validated using **Lucas–Kanade optical flow**.

- Forward optical flow is computed
- Backward flow is computed for consistency checking
- Matches with large displacement or high forward–backward error are rejected

This step removes incorrect matches caused by noise or sudden motion.

6. Visualization

The system provides multiple real-time visual outputs:

- Gaussian filtered frame
- Harris corner visualization

- SIFT keypoint visualization
- Optical flow vectors
- Final feature matching result

An optional dashboard view combines all processing stages into a single window along with tracking statistics.

7. Results and Performance

The implemented system successfully tracks objects across video frames under:

- Noise
- Blur
- Moderate object motion

Key observations:

- Harris filtering improves feature reliability
- Optical flow validation reduces false matches
- Combined techniques provide better robustness than individual methods

8. Limitations

- High computational cost due to SIFT and optical flow
- Performance degrades under heavy occlusion
- Tracking accuracy reduces for very fast motion

9. Conclusion

This project presents a robust video object tracking system that integrates **image filtering, SIFT feature detection, Harris corner filtering, and optical flow validation**. The combination of spatial feature matching and temporal motion consistency results in accurate and stable object tracking across video frames.

The modular design allows easy parameter tuning and future extensions.

10. Future Enhancements

- Replace SIFT with faster feature descriptors
- Introduce bounding box-based object tracking
- Extend the system to multi-object tracking
- Integrate deep-learning-based feature extraction